Aerial Robotics Kharagpur Selection Task – Perception

Task 2.2: Noise Filtering

Harsh Raj, First Year Undergraduate, B. Tech, Computer Science & Engineering

***Abstract* - This documentation contains my solution to the noise filtering problem. Here, the objective was to filter the noise from two of the given inputs. I have used the openCV and numpy library for this. For the given input I tried many classical filtering techniques - mainly morphological transformations and blurs. Different filters or blur work on specific cases based on noise type. Here I have used opening or median blur for iron_man_noisy image and Median Blur and Non-Local mean on landscape_noisy image. Using these I have got the solution to the problem.**

Clearly, Noise is a common problem in video inputs, so we can apply these methods we have learnt in other projects. Like even in task 1 normally the input is not so good, so we must denoise the input before get to work on the actual problem. In creating autonomous aerial robots too, it is necessary to process the visual input which in turn requires us to denoise the image.

I. INTRODUCTION

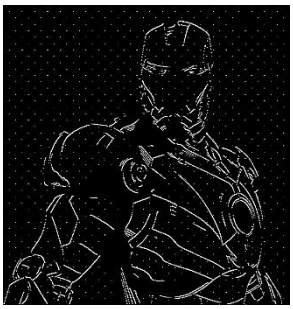
The problem statement was to denoise two given images as inputs by finding the best parameters and methods that are working. We were provided 2 corrupted image files and the goal was to recover clean, usable versions of each using classical image processing techniques available in OpenCV and NumPy.

Image 1 (iron_man_noisy):} This image contained high-frequency white pixel noise scattered across the image. Here, morphological transformation such as Opening, or Median Blur, both worked effectively. These methods are well-suited for removing isolated noise pixels while preserving the underlying structure of the line drawing.

Image 2 (landscape_noisy):} This image also contained salt noise, which was first addressed using Median Blur. However, after removing the impulsive noise, residual color noise remained in the image. To handle this, Non-Local Means (NLM) denoising was applied, which works by averaging pixel values based on patch similarity across the entire image, making it effective for smoothing out color-level noise while retaining fine details.

II. PROBLEM STATEMENT

The problem statement was to denoise two given images as inputs by finding the best parameters and methods that are working. We were provided 2 corrupted image files:



Iron-Man Noisy Image

Landscape Noisy Image

The core challenge in both cases was finding the right balance between noise removal and feature preservation.

III. INITIAL ATTEMPTS

Initially I tried multiple morphological transformations and blurs on the image after learning about them, and understood why the one that is working works and why others do not. On the first image gaussian blur did not work as it blurred the edges too against the preservation of the actual thing. I tried median blur with different sized kernels too. Larger kernels destroy the sketch while smaller ones do not remove the dots. Applying the mid ones with iterations worked the best. In applying opening too, normal kernels work bad as the probability of getting eroded is too high when erosion is applied, so I used cross shaped kernel which worked better.

IV. FINAL APPROACH

Image 1: Iron Man (iron_man_noisy)

Two methods were tried for this image, and both worked effectively:

Method 1 - Median Blur: Median Blur was applied twice with a kernel size of 3. The kernel acts as a mask over which the median of all pixel values is computed. A larger kernel size removes more noise but also risks removing sketch pixels, since the line-width is of a comparable range.

Applying it twice with a small kernel of size 3 strikes the best balance - it removes most of the noise while keeping the sketch pixels intact.

The reason Median Blur works here is that in noisy regions, only one or two pixels within the kernel are 255 (white), so the median evaluates to 0. In sketch regions however, more pixels within the kernel are 255, so the median evaluates to 255 - effectively removing noise while preserving the drawing.

Method 2 - Morphological Opening: Opening is a two-step morphological operation: erosion followed by dilation. Erosion: sets all pixels in the kernel to 0 if any pixel in the kernel is 0. This removes isolated noise dots while retaining sketch pixels, since sketch lines are connected and have a width greater than the kernel size. Dilation: then restores any sketch outlines that were slightly eroded, by setting all pixels to 255 if any pixel in the kernel is 255.

A cross-shaped kernel was used instead of an all-ones kernel, because the all-ones kernel increases the probability of removing sketch pixels. The cross-shaped kernel reduces this risk while still being large enough to eliminate isolated noise dots during erosion.

Image 2: Landscape (landscape_noisy)

A two-step method was used for this image:

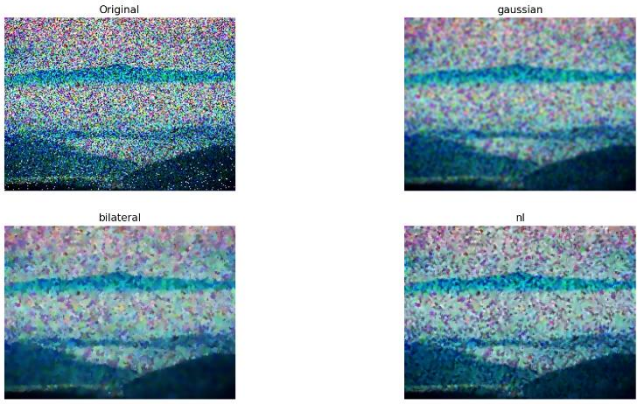
Step 1 - Median Blur: Applied first to remove the impulsive salt noise, like the approach used for Image 1.

Step 2 - Non-Local Means (NLM) Denoising: After Median Blur, residual color noise remained. NLM denoising was applied to address this. NLM works by averaging pixel values based on patch similarity across the entire image - pixels with similar surrounding patches contribute more to the average. This makes it highly effective at smoothing color-level noise while retaining fine structural details in the landscape.

V. RESULTS AND OBSERVATION

Iron Man Denoised Image

The result is such that the first image is denoised successfully, no salt noise remained after median blur is



Landscape Denoised Image

applied with kernel of 3x3 twice, whereas in the case of second image, median blur removes the white salt noise but the RGB noise remains, even after using all the blurs not much improvement was found, however the originality is mostly restored in the case of non-local mean.

VI. FUTURE WORK

I would like to learn about more algorithms to improve the second result. I would also like to apply these in real life projects.

VII. CONCLUSION

It was fun as well as challenging doing these task and getting to know about different algorithms in opencv.

References

[1] OpenCV Documentation.

[2] OpenCV Tutorial byProgrammingKnowledge.